

# An AI-Powered Conversational SQL Tutoring System with Explanation-Driven Feedback

Shashwat Gautam  
Department of Computer Science  
Boise State University  
Email: shashwatgautam@u.boisestate.edu

**Abstract**—This paper proposes an AI-powered, full-stack Conversational SQL Tutoring System that helps students learn SQL through natural language interaction and explanation-driven feedback. Unlike black-box query generators, the system prioritizes understanding: it provides step-by-step explanations of query logic, common mistakes, and alternative solutions. Users can describe problems in English or submit SQL and receive feedback tailored to their expertise. The system combines pretrained language models with rule-based SQL validation in a sandboxed backend and an adaptive explanation layer grounded in Explainable AI (XAI). We outline the problem, related work, method, dataset, timeline, course and PLO mapping, challenges, and expected outcomes for this graduate capstone project.

## I. INTRODUCTION

### A. Problem and Motivation

Many existing SQL tools and AI-based query generators act as “black boxes,” returning answers without explaining why they are correct or incorrect. This leads to shallow learning and over-reliance on automation. The proposed system addresses this gap by functioning as an **intelligent tutoring system** whose primary goal is to teach SQL concepts like joins, subqueries, aggregation, and filtering, through interactive dialogue and reasoning-based explanations.

### B. Beneficiaries

The system targets **students and learners** in database and data science courses, as well as professionals seeking to deepen their SQL understanding. Educators benefit from a tool that scales personalized feedback without replacing human instruction.

### C. Novelty

The novelty lies in an **explanation-first, learner-centered design**. While SQL chatbots and NL-to-SQL systems exist, few prioritize (1) adaptive explanations by proficiency level, (2) mistake-aware feedback that explains *why* a query is wrong, and (3) conceptual learning (how and why) over mere answer generation. This project contributes a human-centered AI system that supports deeper learning through applied NLP and XAI.

## II. RELATED WORK

**Text-to-SQL & NL2SQL.** Surveys [1], [2] show that LLM-based text-to-SQL has advanced significantly, but focus is on

accuracy and generalization, not on pedagogical explanation or error diagnosis for learners.

**SQL Intelligent Tutoring.** SQL-Tutor [3] and ViSQL [4] use constraint-based modeling and semantic feedback. They demonstrate that high-level pedagogical advice improves learning. However, they do not integrate modern NL2SQL or natural language dialogue for problem description, nor do they emphasize adaptive explanation depth or open-vocabulary interaction.

**Gaps.** (1) Existing NL2SQL systems rarely explain *why* a generated query is correct or suggest alternatives. (2) ITSs for SQL often use fixed feedback templates rather than natural language explanations. (3) Few systems combine: natural language problem input, user-written SQL critique, and proficiency-adapted explanations in one platform. Our system targets these gaps with a hybrid, explanation-first architecture.

## III. METHOD

### A. Architecture

The system is a full-stack web application: (1) **Frontend:** conversational UI, SQL editor, and explanation panel. (2) **Backend:** safe sandboxed execution (e.g., SQLite in-memory or containerized DB), request routing, and session management. (3) **AI layer:** intent detection, semantic parsing, NL-to-SQL when the user describes a problem in English, and SQL analysis when the user submits code. (4) **Explanation module:** generation of step-by-step reasoning, common mistakes, and alternatives using a pretrained open-source LLM with prompting and/or light fine-tuning, aligned with XAI principles (faithful, human-readable justifications).

### B. AI & NLP Components

We use **NLP** for: intent detection (e.g., “write query” vs “explain my query”), semantic parsing, and NL-to-SQL translation. We adopt a **hybrid** approach: pretrained open-source language models for generation and explanation, plus **rule-based** SQL validation (syntax, schema consistency, safety) to detect errors and trigger targeted feedback. No large-scale custom training is required; we rely on prompting and optional adapter/LoRA tuning for explanation style.

### C. Explainability and Adaptation

Explanations are generated in natural language (what each clause does, why a join is needed, what a typical mistake

would be). We adapt **explanation depth** by user level: simplified analogies for beginners, concise technical reasoning for advanced users, using level inferred from dialogue or user setting.

#### IV. DATASET AND RESOURCES

We use the **Spider** [5] text-to-SQL benchmark (10k+ questions, 5.6k+ unique SQL queries, 200 databases across 138 domains) for schema diversity and to derive practice problems and reference solutions. Optionally we use **Spider 2.0** [6] for more complex workflows. We may curate a small set of **common misconceptions** (e.g., misuse of GROUP BY, JOIN conditions) from literature and instructor experience to drive mistake-aware feedback. No proprietary or sensitive data is required; all resources are public.

#### V. TIMELINE

TABLE I  
PROPOSED SEMESTER TIMELINE (HIGH-LEVEL).

| Phase               | Weeks | Deliverables                      |
|---------------------|-------|-----------------------------------|
| Literature & design | 1–2   | Proposal, architecture doc        |
| Backend & sandbox   | 3–4   | DB sandbox, API, safety           |
| NL2SQL & analysis   | 4–6   | Intent, NL-to-SQL, SQL checker    |
| Explanation & UI    | 6–7   | Explanation module, frontend      |
| Integration & eval  | 7–9   | End-to-end tests, user study prep |
| Report & demo       | 9–10  | Final report, demo, presentation  |

#### VI. COURSE PREPARATION AND PLO MAPPING

##### A. Course Mapping

The project draws on: **Database systems** (SQL, schemas, query execution), **Machine Learning / NLP** (language models, intent detection, semantic parsing), **Software Engineering** (full-stack design, APIs, testing), and **AI / HCI** (explainability, user-centered design). Courses in NLP, databases, and software engineering provide the necessary foundation for implementation and evaluation.

##### B. PLO Mapping (MS in CS)

Deliverables are mapped to program learning outcomes: **Analysis & design** (architecture, requirement analysis); **Implementation** (full-stack system, AI integration); **Evaluation** (user feedback, accuracy of NL2SQL and explanations); **Communication** (written report, presentation); **Ethics & safety** (sandboxing, responsible AI).

#### VII. CHALLENGES, BACKUP PLAN, AND EXPECTED OUTCOME

##### A. Challenges

(1) **Explanation quality**: LLM-generated explanations may be vague or incorrect. Mitigation: structured prompts, rule-based anchors for known error types, and optional human-in-the-loop review for a subset of feedback. (2) **Schema complexity**: Very large schemas may strain NL2SQL. Mitigation: focus on Spider-sized schemas and limit context to relevant

tables. (3) **Scope creep**: Mitigation: strict MVP (SELECT-only, single-DB sandbox, one LLM backend) and phased feature addition.

##### B. Backup Plan

If explanation quality or NL2SQL accuracy is insufficient: (A) simplify to **template-based explanations** keyed by error codes and query patterns; (B) reduce open-vocabulary input to a **menu-driven** problem selection plus SQL submission only; (C) prioritize **SQL critique and fix suggestions** over full NL-to-SQL. The core contribution (explanation-first, mistake-aware tutoring) remains viable with reduced reliance on free-form generation.

##### C. Expected Outcome

A **working prototype**: web-based conversational SQL tutor with SQL editor, sandboxed execution, NL-to-SQL for described problems, feedback on user-submitted SQL with explanations, and basic proficiency adaptation. Deliverables: (1) deployable application, (2) short user study or expert evaluation, (3) final written report and presentation. The project demonstrates applied AI, NLP, and full-stack engineering at a graduate capstone level and provides an extensible base for future work (e.g., multi-dialect SQL, richer student modeling).

#### REFERENCES

#### REFERENCES

- [1] Y. Sun, B. Tang, and W. Zhang, “A survey of text-to-SQL in the era of large language models,” *arXiv preprint arXiv:2408.05109*, 2024.
- [2] “Rethinking data in NL2SQL: A survey of what we have and what we expect,” *Springer*, 2025. [Online]. Available: <https://link.springer.com/article/10.1007/s44336-025-00026-9>
- [3] T. Mitrović, B. Martin, and M. Mayo, “Using evaluation to shape ITS design: Results and experiences with SQL-Tutor,” *Int. J. Artif. Intell. Educ.*, vol. 12, pp. 243–279, 2002.
- [4] S. Chen et al., “ViSQL: An intelligent online SQL tutoring system,” in *Proc. IEEE Int. Conf. Teaching, Assessment, and Learning for Engineering (TALE)*, 2022, pp. 1–6.
- [5] T. Yu et al., “Spider: A large-scale human-labeled dataset for complex and cross-domain semantic parsing and text-to-SQL task,” in *Proc. EMNLP*, 2018, pp. 3911–3921.
- [6] Spider 2.0 Benchmark. [Online]. Available: <https://spider2-sql.github.io/>
- [7] “Explainability of large language models: Opportunities and challenges toward generating trustworthy explanations,” *arXiv preprint*, 2025.